# Practical lab report

Local inference, Ollama API, Python, RAG and web application

## Scope

This report answers the lab questions in English. The application uses Flask, Ollama, local course documents, keyword retrieval and per-session conversation history. The two-model comparison uses results saved by the notebook on the target computer.

## Observed environment

Ollama 0.40.11 responded through its local HTTP API. The downloaded models are qwen3.5:latest (9.0B) and gemma3:latest (3.9B), both quantized Q4_K_M. The saved benchmark reports no VRAM use for either model.

## Q1

Local AI runs an AI model on a user's computer or on a server the user controls.

## Q2

A cloud LLM processes requests on a provider's infrastructure over the Internet; a local LLM uses local hardware. Capability, cost, and speed depend on the model and hardware.

## Q3

Ollama downloads and manages models, loads them for inference, and provides a local HTTP API.

## Q4

Quantization stores weights with fewer bits. It reduces disk and memory requirements, possibly at some cost to accuracy.

## Q5

More parameters and more bits per parameter require more memory for weights. Context and inference also consume RAM or VRAM.

# Concepts and application

### Q6

A smaller model may fit available memory, answer faster, and use less energy while providing adequate quality.

### Q7

Benefits include data control, offline use after setup, no per-token API charges, and control over deployment.

### Q8

Limitations include hardware, storage, energy, setup, and maintenance requirements; quality may be below that of the strongest cloud models.

### Q9

The browser sends a question to Flask. Python retrieves relevant documents, adds context and conversation history, calls Ollama, and returns the answer, sources, and response time.

### Q10

The REST API connects Python to Ollama through HTTP requests and JSON responses, including /api/chat and /api/generate.

# Retrieval, models and security

## Q11

RAG retrieves relevant documents and adds them to the model's input to ground the answer. This project's keyword retriever may miss synonyms.

## Q12

A text-only LLM processes text; a multimodal model can also accept images or other data types, depending on its capabilities.

## Q13

On the same computer, gemma3 answered the code prompt in 133.1 s versus 144.4 s for qwen3.5. In the cold-start benchmark, gemma3 took 93.06 s and generated 804 tokens at 10.11 tokens/s; qwen3.5 took 169.14 s and generated 739 tokens at 4.93 tokens/s. Both generated prime-number functions passed tests from -5 through 2000. The qwen3.5 implementation also handled 10**400, while gemma3 raised OverflowError because it uses a floating-point square root.

## Q14

I would deploy gemma3 on a computer with limited RAM for this course assistant: it occupied 2.89 GB when loaded, compared with 6.19 GB for qwen3.5, and was faster in the measured benchmark. For code that must support extremely large integers, I would use or adapt the qwen3.5 prime-checking implementation instead.

## Q15

Exposing an unprotected local LLM service may allow unauthorized requests, consume CPU/GPU resources, and expose prompts or documents. Keep it bound to localhost. For remote access, use a firewall and a reverse proxy with authentication and TLS. Treat retrieved documents as data because they may contain prompt-injection instructions.

# Measured model comparison

### qwen3.5:latest

9.0B parameters; Q4_K_M; 6.55 GB on disk; 6.19 GB loaded. Cold load: 16.14 s. Total response: 169.14 s. It generated 739 tokens at 4.93 tokens/s.

### gemma3:latest

3.9B parameters; Q4_K_M; 3.35 GB on disk; 2.89 GB loaded. Cold load: 10.94 s. Total response: 93.06 s. It generated 804 tokens at 10.11 tokens/s.

### Code check

Both generated prime-number functions passed tests from -5 to 2000. The qwen3.5 function handled 10**400. The gemma3 function raised OverflowError on that value because it converts an integer to a floating-point square root.

### Interpretation

Gemma3 used less loaded memory and was faster in this single cold run, so it is the practical limited-RAM choice for the course assistant. Qwen3.5 produced the more robust prime-checking implementation.

### Limits and evidence

These figures come from one cold benchmark run per model. The system-RAM percentage is a whole-machine reading. All six requested screenshots are present in the screenshots directory.

# Benchmark table

Same reinforcement-learning prompt; one cold run per model.

| Measure | qwen3.5:latest | gemma3:latest |
|---|---|---|
| Parameters | 9.0B | 3.9B |
| Quantization | Q4_K_M | Q4_K_M |
| Disk size (GB) | 6.55 | 3.35 |
| Loaded size (GB) | 6.19 | 2.89 |
| VRAM (GB) | 0.0 | 0.0 |
| Load time (s) | 16.14 | 10.94 |
| Response time (s) | 169.14 | 93.06 |
| Generated tokens | 739 | 804 |
| Tokens/s | 4.93 | 10.11 |
| Code prompt time (s) | 144.4 | 133.1 |
| Prime test (-5 to 2000) | Pass | Pass |
| 10**400 code test | Pass | OverflowError |

Loaded size comes from Ollama /api/ps. System RAM percentages were not used as model memory.

Quality scores require reading the generated responses; the performance benchmark saved counters.